EXP-4 IMPLEMENTATION OF A CONVOLUTIONAL NEURAL NETWORK

AIM:

To implement a convolutional neural network (CNN) using PyTorch to classify images from the CIFAR-10 dataset and visualize the learned filters.

SOFTWARE REQUIREMENTS:

* Python 3.x
* PyTorch
* Torchvision
* Matplotlib
* CIFAR-10 dataset

ALGORITHM:

* Import the required libraries
* Load and preprocess the CIFAR-10 dataset
* Define the CNN architecture with convolutional, pooling, and fully connected layers
* Initialize the model and select the adam optimizer
* Train the CNN using cross-entropy loss
* Evaluate the model using test data
* Calculate the classification accuracy
* Visualize the learned filter of the first convolutional layer.

PROGRAM:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt.
```

```
transform = transforms.Compose([
    transforms.ToTensor(), transforms.Normalize(
        (0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5) )
])

trainset = torchvision.datasets.CIFAR10(
    root='./data', train=True,
    download=True, transform=transform)

testloader = torch.utils.data.DataLoader(
    testset, batch_size=64, shuffle=False)

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.relu = nn.ReLU()

device = torch.device(
    "cuda" if torch.cuda.is_available() else "CPU"
)
model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(
    model.parameters(), lr=0.001
)

for epoch in range(10):
    runningloss = 0.0
    model.train()

model.eval()
correct, total = 0, 0

with torch.no_grad():
    for images, labels in testloader:
        images, labels = image.to(device), labels.to(device)
        outputs = model(images)
        predicted = outputs.argmax(dim=1)
```

Expected OUTPUT

Epoch 1, loss: ...
Epoch 2, loss: ...
...
Epoch 10, loss: ...
Test Accuracy: ... %

```
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
print(f"Test accuracy: {100 * correct / total:.2f}%")


filters = model.conv1.weight.detach().cpu()
fig, axes = plt.subplots(1, 8, figsize=(15, 4))
for i in range(8):
    f = filters[i].permute(1, 2, 0)
    f = (f - f.min()) / (f.max() - f.min() + 1e-8)
    axes[i].imshow(f)
    axes[i].set_title(f"filter {i+1}")
    axes[i].axis("off")
plt.show()
```

RESULT:

Thus, a convolutional Neural network was implement using PyTorch and trained on the CIFAR-10 dataset the model was evaluted using test accuracy, and the learned filters were visualize successfully.